Module 9 – Introduction to React.js

THEORY EXERCISE :

Question 1: What is React.js? How is it different from other JavaScript frameworks and libraries?

- React.js is a JavaScript library used for building user interfaces (UI).
- It is mainly used for single-page applications (SPA).
- Developed by Facebook (now Meta).
- Helps in creating reusable UI components.

Feature	React.js	Other Frameworks (Angular, Vue)
Type	Library	Framework
Flexibility	High	Less flexible
Learning Curve	Easy to moderate	Moderate to difficult
DOM Handling	Virtual DOM	Real DOM (or different methods)
Architecture	Component-based	MVC / MVVM

Question 2: Explain core principles of React

1. Virtual DOM

- React uses a Virtual DOM (Document Object Model).
- It is a lightweight copy of the real DOM.
- When data changes:
    - React updates the Virtual DOM first
    - Then compares it with the real DOM (diffing algorithm)
    - Only updates the changed parts → faster performance

2. Component-Based Architecture

- UI is divided into small reusable components.
- Each component:
    - Has its own logic and structure
    - Can be reused anywhere Unidirectional Data Flow
    - Data flows one-way (parent → child).
    - Makes debugging easier and code predictable.

    - 4. JSX (JavaScript XML)
    - Allows writing HTML inside JavaScript.
    - Makes UI code more readable and easy to write.
    - Example:
- Navbar component
- Footer component
- Card component

Reusability

- Components can be reused multiple times → saves time and effort.

Question 3: Advantages of using React.js

Key Advantages:

- Fast Performance (Virtual DOM)
- Reusable Components
- Easy to Maintain Code
- Supports Mobile App Development (React Native)
- Strong Community Support
- Efficient Rendering

- Rich Ecosystem (Redux, Router, etc.)
- Easy Testing

JSX (JavaScript XML)

THEORY EXERCISE

Question 1: What is JSX in React.js? Why is it used?

What is JSX?

- JSX (JavaScript XML) is a syntax extension for JavaScript.
- It allows you to write HTML-like code inside JavaScript.
- Used in React.js to design UI.

Why is JSX used?

- Makes code more readable and understandable
- Combines UI + logic in one place
- Faster development
- Helps React create elements easily

Question 2: How is JSX different from regular JavaScript? Can you write JavaScript inside JSX?

Feature	JSX	JavaScript
Syntax	HTML-like	Pure JS syntax
Readability	Easy for UI	Less intuitive for UI
Usage	UI rendering	Logic & functionality
Compilation	Needs transpiler (Babel)	Direct execution

Can we write JavaScript inside JSX?

YES

Question 3: Importance of Curly Braces {} in JSX

Why {} are used?

- To embed JavaScript expressions inside JSX

What can be written inside {}?

- Variables
- Expressions
- Functions
- Calculations

LAB EXERCISE

Task: Create a React component using JSX

```
function JSXExample() {

  const topic = "JSX";

  const description = "JSX allows us to write HTML inside JavaScript and makes
UI development easier.";



  return (

    <div>

      <h1>Welcome to JSX</h1>

      <p>{description}</p>

    </div>

  );

}



export default JSXExample;
```

```
import JSXExample from "./JSXExample";


function App() {
  return (
    <div>
      <JSXExample />
    </div>
  );
}


export default App;
```

Props and State

Question 1: What are props in React.js? How are props different from state?

What are Props?

- Props (Properties) are used to pass data from parent to child components.
- They are read-only (immutable).
- Help make components dynamic and reusable.

Feature	Props	State
Meaning	Data passed from parent	Data managed inside component
Mutability	Immutable (cannot change)	Mutable (can change)
Control	Parent component	Own component
Usage	Communication	Dynamic behavior

Question 2: Explain State in React

What is State?

- State is a built-in object used to store dynamic data in a component.

- It allows components to update and re-render when data changes.

Question 3: Why is this.setState() used in class components?

Purpose:

- Used to update the state in class components

Lab exercise :

```
function UserCard(props) {
  return (
    <div style={{
      border: "1px solid black",
      padding: "15px",
      width: "200px",
      borderRadius: "10px"
    }}>
      <h2>{props.name}</h2>
      <p>Age: {props.age}</p>
      <p>Location: {props.location}</p>
    </div>
  );
}

export default UserCard;
import UserCard from "./UserCard";
```

```
function App() {
  return (
    <div>
      <UserCard name="Aditi" age={21} location="Ahmedabad" />
      <UserCard name="Rahul" age={25} location="Mumbai" />
    </div>
  );
}

export default App;
```

Task 2: Counter Component

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h1>Count: {count}</h1>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
```

```
  );
}

export default Counter;
import Counter from "./Counter";

function App() {
  return (
    <div>
      <Counter />
    </div>
  );
}

export default App;
```

Routing in React (React Router)

Question 1: What is React Router? How does it handle routing in SPAs?

What is React Router?

- React Router is a library used for routing in React applications.
- It allows navigation between different components/pages without reloading the page.

How it works in Single Page Applications (SPA):

- In a normal website → page reload happens

- In React SPA:
    - Only URL changes
    - React dynamically updates the component view
    - No full page reload → faster performance

Question 2: Difference between Router Components

1. Browser Router

- Wraps the entire app
- Uses HTML5 history API
- Enables routing functionality

2. Route

- Defines which component to show for a path

3. Link

- Used for navigation without page reload

4. Switch (Old) / Routes (New)

- Switch (React Router v5) → renders first matching route
- Routes (React Router v6) → replacement of Switch

Lab exercise

Task 1: Basic Routing (Home & About)

```
// Home.js

function Home() {

  return <h1>Home Page</h1>;

}

export default Home;

// About.js

function About() {
```

```
  return <h1>About Page</h1>;
}
export default About;
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Home from "./Home";
import About from "./About";

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Home from "./Home";
import About from "./About";
```

```
function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

Task 2: Navigation Bar with Link

```
// Contact.js
function Contact() {
  return <h1>Contact Page</h1>;
}
export default Contact;
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";
import Home from "./Home";
import About from "./About";
import Contact from "./Contact";
```

```
function App() {
  return (
    <BrowserRouter>

      <nav>
        <Link to="/">Home</Link> |{" "}
        <Link to="/about">About</Link> |{" "}
        <Link to="/contact">Contact</Link>
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>

    </BrowserRouter>
  );
}

export default App;
```

State Management (Redux, Redux-Toolkit or Recoil) :

Question 1: What is Redux? Why is it used?

What is Redux?

- Redux is a state management library.
- It helps manage global state in large React applications

Why Redux is used?

- Centralized state management
- Predictable state updates
- Easy debugging
- Share data between components easily

Question 2: How Recoil simplifies state management?

What is Recoil?

- Recoil is a modern state management library by Facebook

Feature	Redux	Recoil
Setup	Complex	Simple
Boilerplate	High	Low
Learning Curve	Higher	Easier
State Access	Global store	Direct access via atoms
Code Size	More	Less

LAB EXERCISE

Task 1: Counter App using Redux:

```
// reducer.js

const initialState = { count: 0 };



function reducer(state = initialState, action) {
```

```
  switch (action.type) {
    case "INCREMENT":
      return { count: state.count + 1 };
    case "DECREMENT":
      return { count: state.count - 1 };
    default:
      return state;
  }
}

export default reducer;
import { createStore } from "redux";
import reducer from "./reducer";

const store = createStore(reducer);
export default store;
import { Provider, useDispatch, useSelector } from "react-redux";
import store from "./store";

function Counter() {
  const count = useSelector((state) => state.count);
  const dispatch = useDispatch();
```

```
  return (
    <div>
      <h1>{count}</h1>
      <button onClick={() => dispatch({ type: "INCREMENT" })}>+</button>
      <button onClick={() => dispatch({ type: "DECREMENT" })}>-</button>
    </div>
  );
}

function App() {
  return (
    <Provider store={store}>
      <Counter />
    </Provider>
  );
}

export default App;
```